Algoritmos de Ordenamiento

Conceptos, Visualizaciones y Ejemplos de Código

Estructura de Datos

Luis Gabriel Rivas Diaz

2026

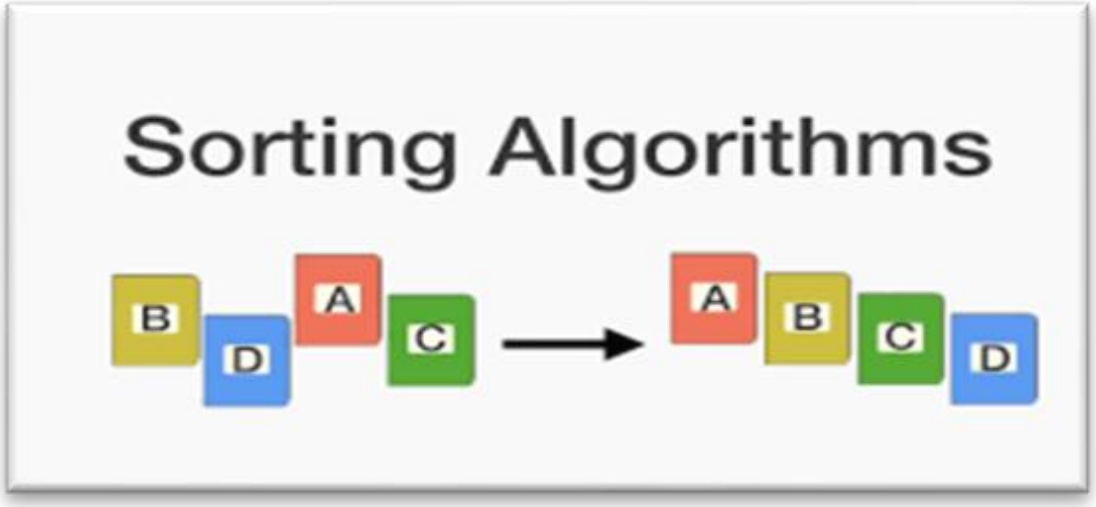
Contenido

1. Introducción
2. Bubble Sort
3. Insertion Sort
4. Selection Sort
5. Shell Sort
6. Quick Sort
7. Merge Sort
8. Radix Sort

¿Qué es un algoritmo de ordenamiento?

Definición General

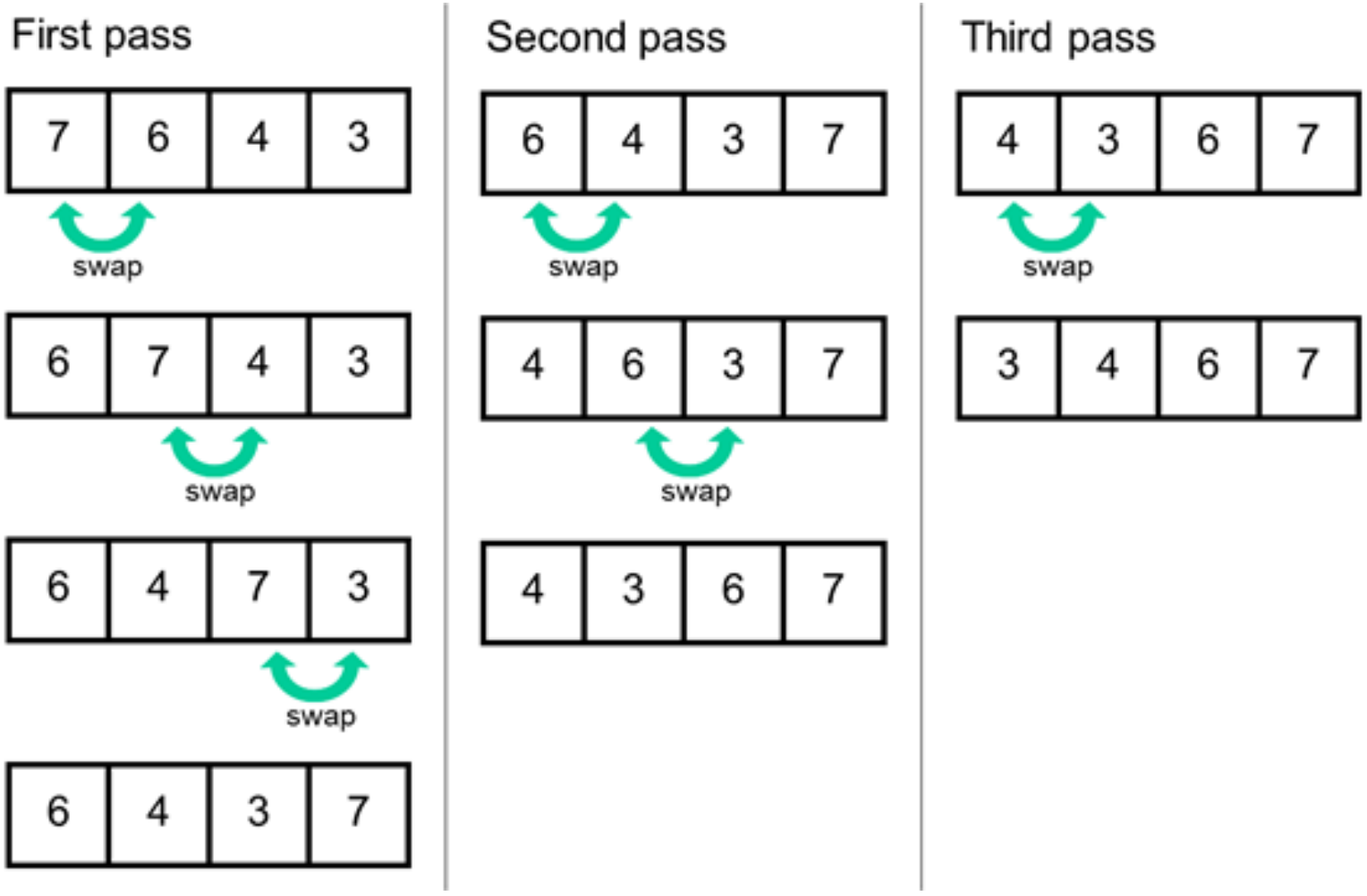
Un algoritmo de ordenamiento reorganiza los elementos de una lista o arreglo según un criterio definido generalmente de menor a mayor mediante operaciones de comparación o mediante procesos especializados que no comparan directamente todos los datos.



Bubble Sort

Definición

Bubble Sort es un algoritmo de comparación simple que revisa repetidamente cada par de elementos adyacentes y los intercambia si están en el orden incorrecto. Este proceso continúa hasta que ya no se requieren intercambios, lo que significa que la lista está ordenada.

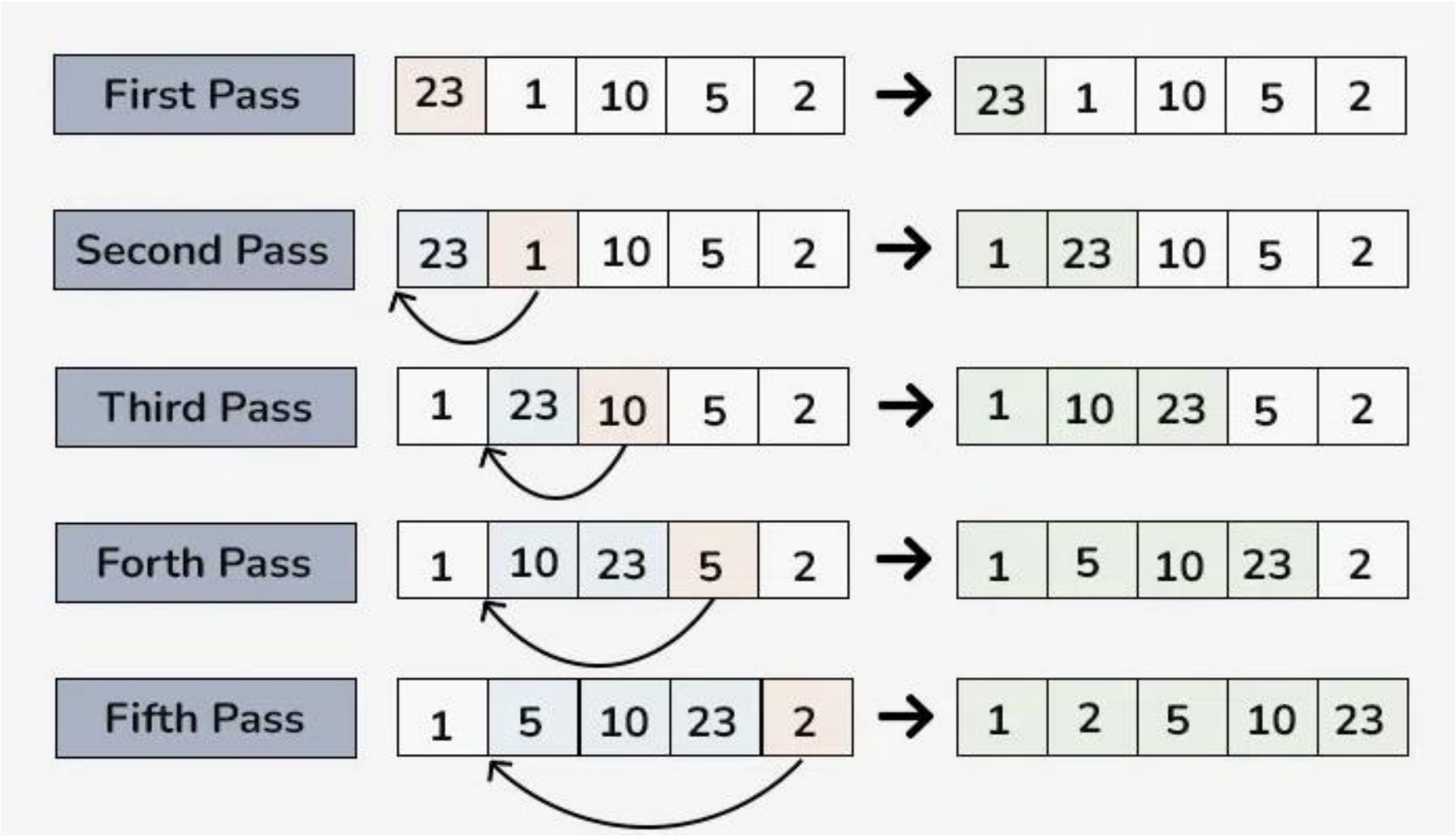

First pass
7 6 4 3
swap
6 7 4 3
swap
6 4 7 3
swap
6 4 3 7
Second pass
6 4 3 7
swap
4 6 3 7
swap
4 3 6 7
Third pass
4 3 6 7
swap
3 4 6 7

```java
public class BubbleSort {

    public static void main(String[] args) {

        int[] numeros = {5, 3, 8, 4, 2};

        int n = numeros.length;

        for (int i = 0; i < n - 1; i++) {

            for (int j = 0; j < n - 1 - i; j++) {

                if (numeros[j] > numeros[j + 1]) {

                    int temp = numeros[j];
                    numeros[j] = numeros[j + 1];
                    numeros[j + 1] = temp;

                }
            }
        }

        System.out.println("Arreglo ordenado:");

        for (int num : numeros) {
            System.out.print(num + " ");
        }
    }
}
```

Insertion Sort

Definición

Insertion Sort construye la lista ordenada tomando un elemento a la vez e insertándolo en la posición correcta de la parte ordenada restante. Es eficiente cuando la lista ya está parcialmente ordenada.

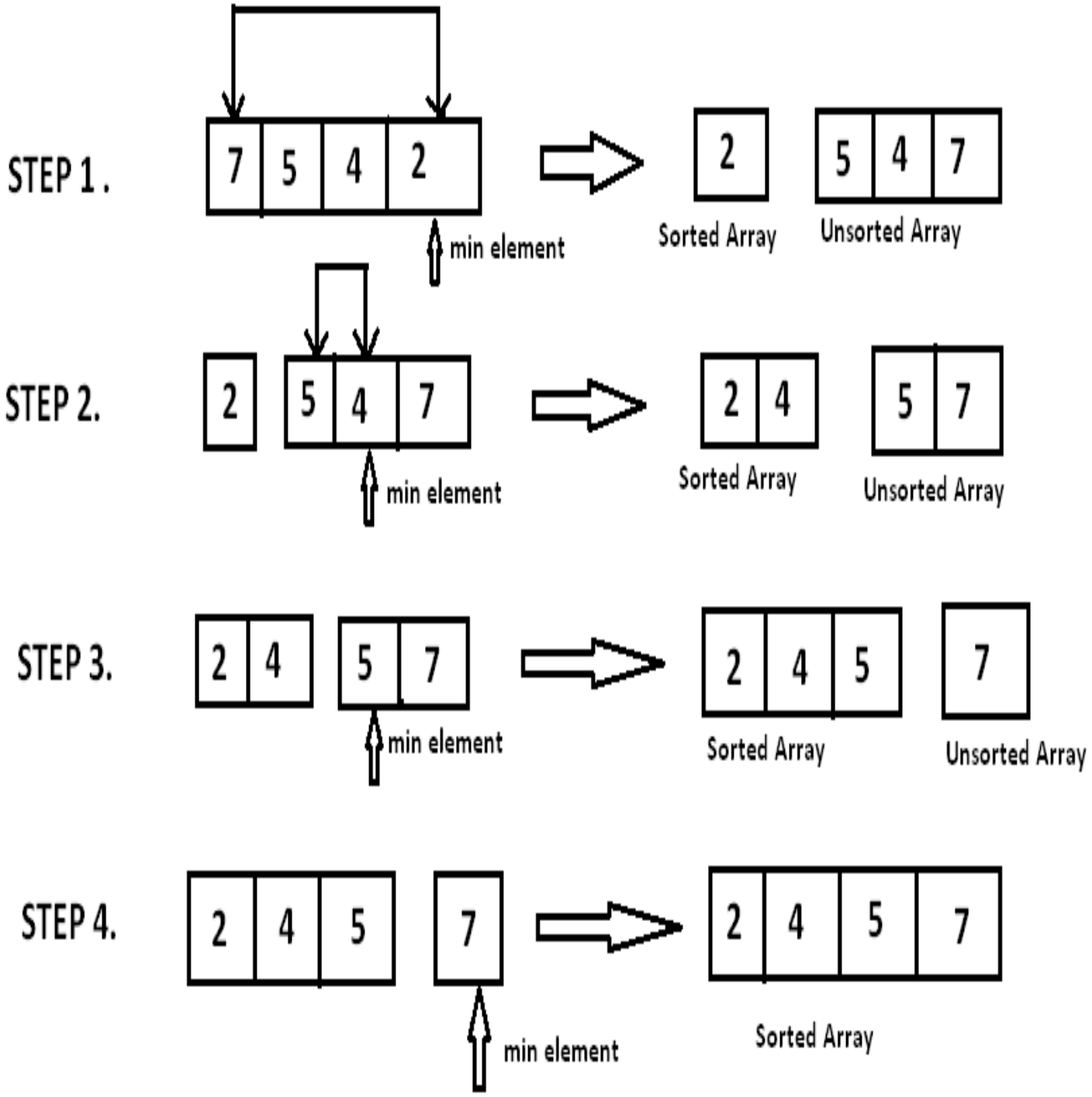

First Pass
23 1 10 5 2
23 1 10 5 2
Second Pass
23 1 10 5 2
1 23 10 5 2
Third Pass
1 23 10 5 2
1 10 23 5 2
Forth Pass
1 10 23 5 2
1 5 10 23 2
Fifth Pass
1 5 10 23 2
1 2 5 10 23

```java
public class InsertionSort {

    public static void main(String[] args) {

        int[] numeros = {5, 3, 8, 4, 2};

        int n = numeros.length;

        for (int i = 1; i < n; i++) {

            int clave = numeros[i];
            int j = i - 1;

            while (j >= 0 && numeros[j] > clave) {
                numeros[j + 1] = numeros[j];
                j = j - 1;
            }

            numeros[j + 1] = clave;
        }

        System.out.println("Arreglo ordenado:");

        for (int num : numeros) {
            System.out.print(num + " ");
        }
    }
}
```

Selection Sort

Definición

Selection Sort divide la lista en dos partes: ordenada y no ordenada. En cada paso selecciona el elemento más pequeño de los no ordenados y lo coloca al final de la parte ordenada.


STEP 1 .
7 5 4 2
min element
2
Sorted Array
5 4 7
Unsorted Array
STEP 2.
2 5 4 7
min element
2 4
Sorted Array
5 7
Unsorted Array
STEP 3.
2 4 5 7
min element
2 4 5
Sorted Array
7
Unsorted Array
STEP 4.
2 4 5 7
min element
2 4 5 7
Sorted Array

```java
public class SelectionSort {

    public static void main(String[] args) {

        int[] numeros = {5, 3, 8, 4, 2};

        int n = numeros.length;

        for (int i = 0; i < n - 1; i++) {

            int min = i;

            for (int j = i + 1; j < n; j++) {

                if (numeros[j] < numeros[min]) {
                    min = j;
                }

            }

            int temp = numeros[min];
            numeros[min] = numeros[i];
            numeros[i] = temp;

        }

        System.out.println("Arreglo ordenado:");

        for (int num : numeros) {
            System.out.print(num + " ");
        }
    }
}
```

Shell Sort

Definición

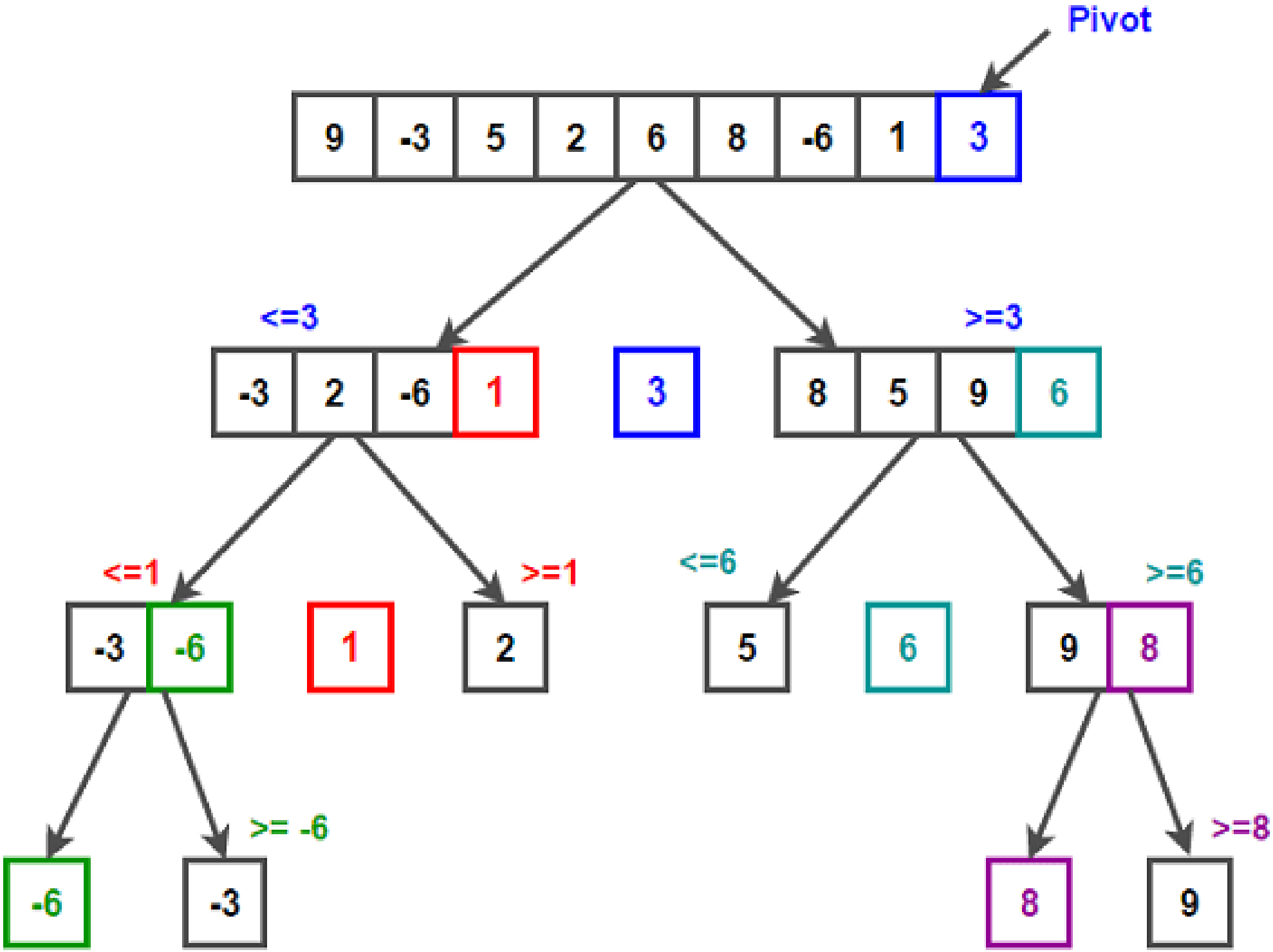
Shell Sort es una mejora de Insertion Sort que ordena pares de elementos separados por distancias (intervalos o gaps) mayores, reduciendo gradualmente el espacio entre comparaciones para mejorar eficiencia.

```java
public class ShellSort {

    public static void main(String[] args) {

        int[] numeros = {5, 3, 8, 4, 2};

        int n = numeros.length;

        for (int gap = n / 2; gap > 0; gap = gap / 2) {

            for (int i = gap; i < n; i++) {

                int temp = numeros[i];
                int j;

                for (j = i; j >= gap && numeros[j - gap] > temp; j = j - gap) {
                    numeros[j] = numeros[j - gap];
                }

                numeros[j] = temp;
            }
        }

        System.out.println("Arreglo ordenado:");

        for (int num : numeros) {
            System.out.print(num + " ");
        }
    }
}
```

Quick Sort

Definición

Quick Sort utiliza la técnica de “divide y vencerás”, eligiendo un pivote y particionando la lista para que los elementos menores queden antes y los mayores después, luego ordena recursivamente cada sublista.

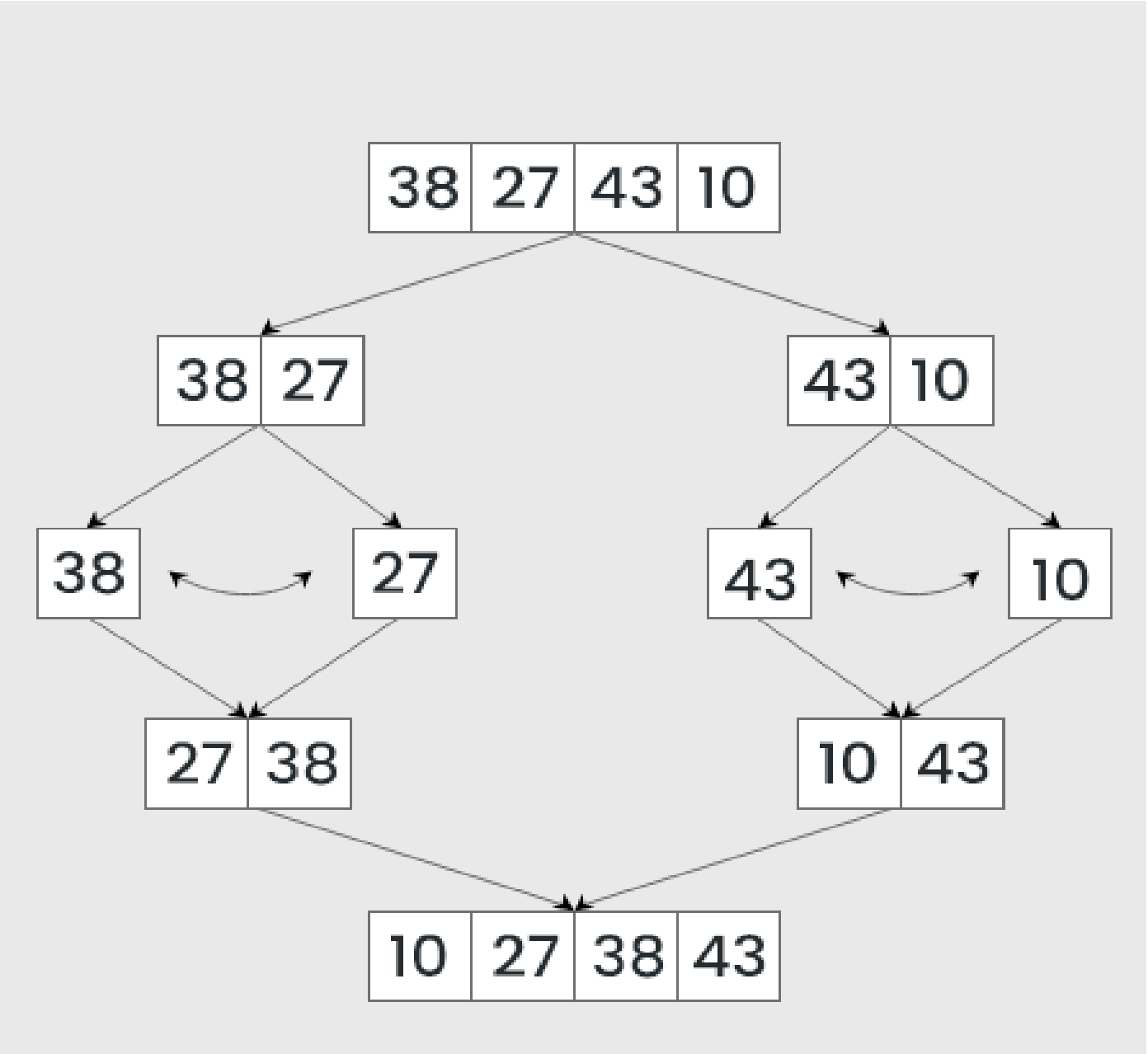

Pivot
9 -3 5 2 6 8 -6 1 3
<=3
-3 2 -6 1
3
>=3
8 5 9 6
<=1
-3 -6
1
>=1
2
<=6
5
6
>=6
9 8
-6
>= -6
-3
8
>=8
9

```java
public class QuickSort {

    public static void quickSort(int[] a, int inicio, int fin) {
        if (inicio < fin) {
            int p = particion(a, inicio, fin);
            quickSort(a, inicio, p - 1);
            quickSort(a, p + 1, fin);
        }
    }

    public static int particion(int[] a, int inicio, int fin) {
        int pivote = a[fin];
        int i = inicio - 1;

        for (int j = inicio; j < fin; j++) {
            if (a[j] < pivote) {
                i++;
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        int temp = a[i + 1];
        a[i + 1] = a[fin];
        a[fin] = temp;

        return i + 1;
    }

    public static void main(String[] args) {
        int[] numeros = {5,3,8,4,2};

        quickSort(numeros, 0, numeros.length - 1);

        for (int n : numeros) {
            System.out.print(n + " ");
        }
    }
}
```

Merge Sort

Definición

Merge Sort también aplica “divide y vencerás”, dividiendo la lista en mitades, ordenando cada una y luego fusionándolas en una sola lista ordenada.

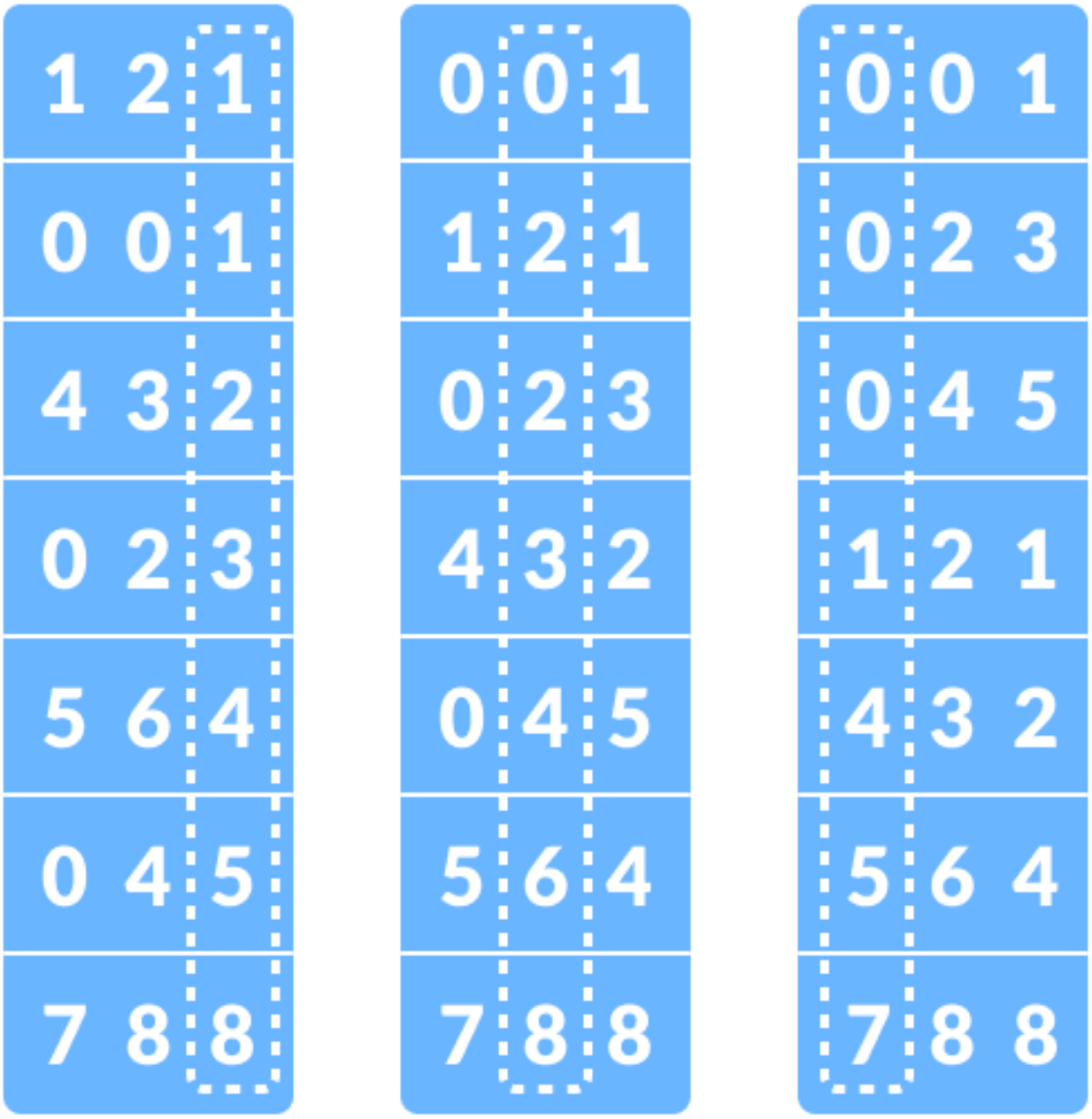

38 27 43 10
38 27
43 10
38
27
43
10
27 38
10 43
10 27 38 43

```java
public class MergeSort {

    static void mergeSort(int[] a, int inicio, int fin) {
        if (inicio < fin) {
            int medio = (inicio + fin) / 2;
            mergeSort(a, inicio, medio);
            mergeSort(a, medio + 1, fin);
            merge(a, inicio, medio, fin);
        }
    }

    static void merge(int[] a, int inicio, int medio, int fin) {
        int[] temp = new int[fin - inicio + 1];
        int i = inicio, j = medio + 1, k = 0;

        while (i <= medio && j <= fin)
            temp[k++] = (a[i] < a[j]) ? a[i++] : a[j++];

        while (i <= medio) temp[k++] = a[i++];
        while (j <= fin) temp[k++] = a[j++];

        for (int x = 0; x < temp.length; x++)
            a[inicio + x] = temp[x];
    }

    public static void main(String[] args) {
        int[] numeros = {5,3,8,4,2};

        mergeSort(numeros, 0, numeros.length - 1);

        for (int n : numeros)
            System.out.print(n + " ");
    }
}
```

Radix Sort

Definición

Radix Sort es un algoritmo no comparativo que ordena elementos procesando sus dígitos individuales, organizándolos primero por posiciones menos significativas y luego por las más significativas.


1 2 1
0 0 1
4 3 2
0 2 3
5 6 4
0 4 5
7 8 8
0 0 1
1 2 1
0 2 3
4 3 2
0 4 5
5 6 4
7 8 8
0 0 1
0 2 3
0 4 5
1 2 1
4 3 2
5 6 4
7 8 8

```java
public class RadixSort {

    static void radixSort(int[] a) {
        int max = a[0];
        for (int n : a) if (n > max) max = n;

        for (int exp = 1; max / exp > 0; exp *= 10) {
            int[] salida = new int[a.length];
            int[] c = new int[10];

            for (int i = 0; i < a.length; i++)
                c[(a[i] / exp) % 10]++;

            for (int i = 1; i < 10; i++)
                c[i] += c[i - 1];

            for (int i = a.length - 1; i >= 0; i--)
                salida[--c[(a[i] / exp) % 10]] = a[i];

            for (int i = 0; i < a.length; i++)
                a[i] = salida[i];
        }
    }

    public static void main(String[] args) {
        int[] numeros = {170,45,75,90,802,24,2,66};

        radixSort(numeros);

        for (int n : numeros)
            System.out.print(n + " ");
    }
}
```

Conclusiones

- Cada algoritmo tiene ventajas y desventajas dependiendo del tamaño y estado de la lista.
- Algoritmos como Quick Sort o Merge Sort ofrecen mejor rendimiento promedio para listas grandes.
- Radix Sort puede ser muy eficiente cuando los datos pueden dividirse por dígitos.